

XACP V1 Preview SDK

Experimental Developer Documentation

Based on ZZ9000OS MNT 1.13

1. Introduction

XACP (Xanxi ARM Coprocessor Protocol) is an experimental architecture designed to turn the ZZ9000's embedded ARM into a genuine application coprocessor for the Amiga. The goal isn't to replace the 68k or emulate the whole Amiga system, but to let AmigaOS offload heavy tasks to the ARM core inside the Zynq. The project started with experiments on the Zynq's Core0 for MP3 decoding and multimedia processing, then gradually expanded to dynamic use of the second ARM core (Core1) as an independent compute engine.

XACP is built on several key ideas: Amiga ↔ ARM communication via shared registers and DDR memory, dynamic bare-metal ARM code execution, persistent application services, and a command protocol that lets AmigaOS control processing on the ARM side. The long-term goal is a generic coprocessor architecture capable of handling tasks like audio/video decoding, decompression, graphics rendering, DSP work, or emulation – all while keeping the native Amiga system running normally on the 68k.

Early XACP experiments were done on Core0 as part of the MP3 project. The idea was to use the ARM as an audio decoding engine running in the background while the Amiga handled the UI and multitasking. Those tests validated ARM ↔ Amiga communication via shared DDR memory and laid the groundwork for an application-level protocol.

Research later moved to Core1. The ZZ9000 firmware sources revealed an internal mechanism that lets Core0 wake up and dynamically control Core1 using hardware reset, a boot vector, and the SEV instruction. That discovery led to a dynamic execution pipeline for bare-metal ARM blobs on Core1. Current experiments have validated dynamic ARM code launch, shared DDR communication, heartbeats, control mailboxes, real-time graphics rendering, and integration with Workbench RTG.

The current roadmap is to gradually connect Core1 to the existing XACP ecosystem. Core0 would keep its role as main controller and interface to the XACP registers visible from the Amiga, while Core1 becomes a resident compute engine that runs heavy tasks on demand. Core0 would act as a bridge between AmigaOS and the permanent Core1 runtime, using a shared DDR mailbox and a command protocol. This approach would progressively turn the ZZ9000's ARM into a true application coprocessor tightly integrated with the Amiga environment.

2. Firmware evolution

XACP Firmware Modifications — From the Early Days to XX16c

Context and Protocol Evolution

Since the start of the XACP project, the ``main.c`` firmware (based on MNT 1.13) has gone through two generations of changes. The first generation used a set of dedicated registers: ``0x62`` for commands, ``0x66`/`0x68`/`0x6A`/`0x6C`/`0x6E`/`0xA6`` for I/O parameters, and ``0x64`/`0xA8`/`0xAA`/`0xAC`/`0xAE`` for read registers. This scheme was dropped during actual development because of Zorro III bus constraints on unaligned UWORD accesses, which made several of those addresses unreliable.

The second generation — a state machine on ``0x64`/`0x68`` — has been the valid architecture since X8 and remained so up to XX10.

Added Global Variables

```
```c
static mp3dec_t xanxi_mp3d;
static int xanxi_initialized = 0;
static uint32_t x_in_off = 0;
static uint32_t x_out_off = 0;
static uint32_t x_in_size = 0;
static uint8_t x_pending_param = 0;
static uint16_t x_status = 0;
static uint16_t x_result = 0;
static uint16_t x_rate = 0;
static uint16_t x_channels = 0;
static uint16_t x_error = 0;
static uint16_t x_debug_hi = 0;
static uint16_t x_debug_lo = 0;
```
```

The streaming pipeline also adds an internal ARM PCM ring buffer with its own state variables (``mp3_avail``, ``mp3_consumed``, ``mp3_fill``, ``mp3_eof``, ``pcm_read``, ``pcm_write``, ``consumed_this_pass``).

Write Registers (``switch(zaddr)``)

****Case ``0x64`` — main command register (state machine)****

| | | | | | |
|--|--|-------|-------|--------|--|
| | <code>`zdata`</code> | value | | Effect | |
| | ----- | | ----- | | |
| | <code>`0x10`–<code>`0x15`</code> Selects the next parameter (written via <code>`0x68`</code>)</code> | | | | |

```
`0`	`CMD_RESET`: `mp3dec_init()`, resets all parameters
`1`	`CMD_DECODE_FRAME`: offline MP3 frame decoding with minimp3
`2`	`CMD_STREAM_FEED`: refills the ARM streaming MP3 buffer
`3`	`CMD_STREAM_POLL`: streaming decode pass + PCM ring fill
```

****Case `0x68` — value of the selected parameter****

Applies `zdata` to the variable designated by `x\_pending\_param`, then sets `x\_pending\_param = 0`.

Codes `0x10`–`0x15` select respectively: `x\_in\_off` HI, `x\_in\_off` LO, `x\_out\_off` HI, `x\_out\_off` LO, `x\_in\_size` HI, `x\_in\_size` LO.

Read Registers

```
Address	Content
`0x64`	`x_status` (0=IDLE, 1=BUSY, 2=DONE, 3=ERROR)
`0xA8`	`x_channels`
`0xAA`	`x_error`
`0xAC`	`x_result` (decoded samples or copied bytes)
`0xAE`	`x_rate` (sample rate)
`0xF6`	`x_debug_hi` (source buffer bytes 0–1 after cache invalidation)
`0xF8`	`x_debug_lo` (source buffer bytes 2–3 after cache invalidation)
```

All registers return their value in the upper 16 bits of a 32-bit word (`data = value << 16`), following the Z3 behaviour of the MNT firmware.

Cache Policy Applied

Before the ARM reads any source buffer: `Xil\_DCacheInvalidateRange()` with strict 32-byte alignment. After the ARM writes to a destination buffer (PCM or copy): `Xil\_DCacheFlushRange()` on the exact range written, never on the whole framebuffer.

Fixes from XX9 to XX10

XX10 is a targeted patch to the streaming pipeline, with no architectural changes. Two bugs were fixed.

****Bug 2 — streaming decode loop condition:****

```
```c
```

```
/* XX9 — mp3_avail never changes, potentially infinite loop */
while (mp3_avail > 4 && pcm_free > ... && frames_this_pass < 8)
```

```
/* XX10 */
while (consumed_this_pass < mp3_avail &&
 (mp3_avail - consumed_this_pass) > 4 &&
 pcm_free > ... && frames_this_pass < 8)
...
```

**\*\*Bug 3 — uint32 underflow in refill threshold calculation:\*\***

```
``c
/* XX9 — underflow if consumed_this_pass >= mp3_avail
 → condition false → refill never triggered → silent starvation */
if ((mp3_avail - consumed_this_pass) < 32768 && !mp3_eof)
```

```
/* XX10 */
uint32_t remaining = (consumed_this_pass <= mp3_avail
 ? (mp3_avail - consumed_this_pass) : 0;
if (remaining < 32768 && !mp3_eof)
...
```

Bug 3 was the main cause of the progressive choppiness toward the end of tracks: the ARM PCM ring would silently drain without ever requesting a refill.

---

Hardware-Validated State at the End of XX10 (Amiga 4000, ZZ9000)

- Offline decode (`ZZMP3Play\_XACP`): perfect — correct ARM decode, correct endianness conversion, correct WAV output, correct AHI playback, 68060 CPU load 2-4%.
- Streaming PCM pipeline (`ZZMP3StreamDump\_XACP`): resulting WAV perfectly clean after little-endian swap on write — refill, leftover, cache flush, PCM ring, streaming endianness all validated.
- Real-time streaming playback (`ZZMP3StreamPlay\_XACP`): music recognisable, residual choppiness under investigation for XX11.

---

What Remains for XX11

The only missing piece for smooth streaming is the **\*\*Amiga-side MP3 ring\*\*** — a mechanism that lets the Amiga continuously pre-feed the ARM's MP3 buffer in advance, without waiting for the buffer to be exhausted before issuing a new `CMD\_STREAM\_FEED`. The ARM architecture (PCM ring, streaming decode, cache policy) is validated and will not be changed.

### 3. XACP Software Inventory

---

#### 1. ZZDecodeTest (v1.0 → v2-2b) — CLI protocol validation tool

The first tool in the project, created to validate Amiga ↔ ARM communication before any real audio development. Several revisions followed.

**\*\*Role:\*\*** Send an MP3 file from Amiga RAM to ZZ9000 DDR, trigger an ARM command, read the status, and save the resulting PCM to `RAM:test.raw`.

**\*\*Protocol used:\*\*** `0x64`/`0x68` state machine (IPC v2). Early versions only tested `CMD\_MEMCPY` to validate the memory path before implementing decoding.

**\*\*Build:\*\***

```
```bat
m68k-amigaos-gcc -O2 -noixemul -m68000 -o ZZDecodeTest ZZDecodeTest.c -lamiga
```
```

**\*\*Success criteria:\*\*** `status=2, result>0`, raw file readable in Audacity as 16-bit signed stereo.

---

#### 2. ZZMP3Play\_XACP — offline MP3 playback via ARM

The first real audio player validated on hardware. **\*\*Offline mode:\*\*** the whole MP3 file is loaded into memory, decoded in one pass by the ARM, then played through AHI.

**\*\*Architecture:\*\***

...

Amiga loads MP3 into RAM

```
→ CopyMem → ZZ9000 DDR (fb + 0x04100000)
→ XACP_Command at fb + 0x04000000, OP_MP3_DECODE = 3
→ ARM minimp3 decodes → PCM to DDR (fb + 0x04800000)
→ Amiga reads PCM, swaps endian LE → AHI → audio out
...
```

**\*\*Validated on A4000 (XX7+):\*\*** correct ARM decoding, correct endian conversion, correct reference WAV, correct AHI playback. 68060 CPU load: **\*\*2–4%\*\*** during playback.

---

#### 3. ZZMP3StreamDump\_XACP — streaming pipeline dump without AHI

A **diagnostic** tool created to isolate streaming bugs without the complexity of AHI. It uses exactly the same streaming pipeline as ZZMP3StreamPlay, but instead of playing audio, it dumps the PCM ring contents to a WAV file on disk.

**Architecture:**

...

Same pipeline as ZZMP3StreamPlay

→ BUT: output = DOS WAV file, not AHI

→ + swap back to LE before writing (the PCM ring is big-endian for AHI)

...

**Key usefulness:** confirmed that the decode/refill/leftover/cache ARM pipeline was correct independently of AHI issues. Result: perfectly clean WAV after the swap fix.

---

#### 4. ZZMP3StreamPlay\_XACP — real-time MP3 streaming playback

Full streaming player with asynchronous AHI double-buffering. Production mode.

**DDR memory layout:**

...

fb + 0x04002000 → struct XACP\_StreamControl (shared ARM/Amiga state)

fb + 0x04100000 → MP3 input ring (512 KB)

fb + 0x04200000 → PCM output ring (1 MB, big-endian for AHI)

...

**Opcodes:** `OP\_STREAM\_OPEN = 4`, `OP\_STREAM\_CLOSE = 5`.

**Amiga side:** asynchronous AHI double-buffer (`SendIO`/`WaitIO`), 128 KB prefill before first send, 64 KB chunks, MP3 refill triggered by ARM `need\_refill`.

**Validated on A4000 (XX10):** music recognisable, residual choppiness fixed by bugs 2+3 in XX10. The Amiga-side MP3 ring (XX11) is the last missing piece for smooth streaming.

---

#### 5. ZZMandelbrot\_XACP — pure compute offload demo

First public demonstration of 68k → ARM compute offloading, unrelated to audio.

**Architecture:**

...

Amiga opens an Intuition RTG window

→ calculates the same Mandelbrot on the 68060 (timing measurement)

→ sends parameters via XACP\_Command (fb + 0x04000000)

OP\_MANDELBROT = 2, Q16.16 parameters, real RTG pitch

→ ARM computes and writes directly to the RTG framebuffer  
→ Amiga displays the result + ARM/68060 speed ratio  
→ validation checksum proves the ARM did the rendering  
...

The `XACP\_Command` structure includes `framebuffer\_ptr`, `width`, `height`, `max\_iter`, and the four fractal bounds `xmin/xmax/ymin/ymax` as Q16.16 fixed-point. The checksum written to the result registers unambiguously proves that the computation was executed by the ARM.

**\*\*Build:\*\***

```
```bat
m68k-amigaos-gcc -O2 -noixemul -m68020 -o ZZMandelbrot_XACP ZZMandelbrot_XACP.c -
lamiga
```
```

---

## 6. mpeg.library (ZZmpeg) — transparent replacement for AmigaAMP

A standard AmigaOS library implementing the full MPEG-A API, backed by XACP. It replaces the original `mpeg.library` without any player-side configuration. This is the end-user product of the XACP audio project.

**\*\*Exposed API:\*\***

...

MPEG\_A\_open() → XACP OP\_STREAM\_OPEN → ARM starts decoding  
MPEG\_A\_decode\_frame() → reads 1152 samples from DDR PCM ring, deinterleaves L/R  
MPEG\_A\_close() → XACP OP\_STREAM\_CLOSE  
MPEG\_A\_seek() → rewind (V1)  
MPEG\_A\_time()  
MPEG\_A\_find\_sync()  
MPEG\_A\_scale()  
...

**\*\*Compatibility:\*\*** AmigaAMP loads this library automatically via `OpenLibrary("mpeg.library", 0)`. No player modifications required. `bs\_access` support is implemented for players that pre-load the file into RAM.

---

## 7. ZZUNZIP — under design (Core1)

ZIP decompressor that offloads DEFLATE to Core1 via a DDR mailbox. The first XACP use case that leverages Core1 as a separate compute engine from Core0.

**\*\*Planned architecture:\*\***

...

ZZUNZIP.c (68k) : parses ZIP, copies compressed data → DDR  
→ mailbox DDR CMD\_INFLATE → Core1 (puff.c bare-metal)  
→ Core1 decompresses → DDR → Amiga reads and writes the file  
...

The chosen DEFLATE implementation is `puff.c` (Mark Adler) — zero malloc, zero dependencies, direct src/dst API. The architecture is validated on paper, but not yet implemented.

## 4. Memory Model / Core 1 Architecture

### ZZ9000 / XACP Memory Model

#### Overview

The ZZ9000's Zynq has:

- DDR as seen by the ARM
- A Zorro III window as seen by the Amiga
- RTG framebuffer
- Free areas usable by XACP

**\*\*Key point:\*\*** The ARM view and the Amiga view are **\*\*not\*\*** the same.

---

#### Main mapping

**\*\*Amiga view (Zorro)\*\***

The Amiga sees ZZ9000 memory as:

`fb + offset`

where `fb = board + 0x00010000`

**\*\*ARM view\*\***

The ARM core sees the plain offset directly.

| Amiga view      | ARM view   |  |
|-----------------|------------|--|
| -----           | -----      |  |
| fb + 0x04100000 | 0x04100000 |  |
| fb + 0x04200000 | 0x04200000 |  |

This is the fundamental rule used by XACP.

---

#### Important memory regions



## **\*\*RTG framebuffer\*\***

Typical start: `0x00200000`

Used by: Picasso96 RTG, screen bitmaps, graphics surfaces.

**\*\*Caution:\*\*** depends on resolution, number of screens, colour depth. Writing without knowing the exact pitch → RTG corruption.

**\*\*Safe Core1 / blob area\*\*** (experimentally validated)

| Usage          | ARM address |
|----------------|-------------|
| -----          | -----       |
| Core1 blob     | 0x04200000  |
| Shared mailbox | 0x04300000  |
| Render buffer  | 0x04340000  |

These areas have been successfully used for: heartbeat, Julia, Mandelbrot, mailbox, double buffering.

---

## **Validated Core1 registers**

| Register | Role       |
|----------|------------|
| -----    | -----      |
| 0x0090   | ARM_RUN_HI |
| 0x0092   | ARM_RUN_LO |

Used by `arm\_app\_run()` in the MNT firmware.

---

## **Real Core1 sequence**

**\*\*Amiga\*\***

1. Load ARM blob at `fb + 0x04200000`

2. Write:

`REG\_ARM\_RUN\_HI = 0x0420`

`REG\_ARM\_RUN\_LO = 0x0000`

**\*\*Core0 firmware\*\***

Performs:

- Reset Core1
- Write boot vector
- SEV
- Wake up Cortex-A9 #1

**\*\*Core1\*\***

Starts at the blob address, then:

- Heartbeat
- Mailbox
- Rendering
- Computation
- Cooperative return

---

### **Validated cache policy**

**\*\*ARM reading DDR written by Amiga\*\***

Always: `Xil\_DCacheInvalidateRange()` with 32-byte aligned address and size.

**\*\*ARM writing DDR visible to Amiga\*\***

Always: `Xil\_DCacheFlushRange()` on the exact range written, never on the whole framebuffer.

---

### **Dangerous areas**

**\*\*Active RTG framebuffer\*\***

Writing into an RTG-used area can cause: artefacts, corruption, freezes, Guru.

**\*\*Unvalidated offsets\*\***

Offsets below `0x04000000` have historically caused: erratic behaviour, RTG collisions, video corruption.

## **5. Current Validated State**

### **XACP / XX Firmware Timeline**

#### **Pre-XACP — base MNT 1.13 firmware**

- Standard MNT ZZ9000 firmware
- No formal coprocessor protocol
- Core1 already present in the sources (`core1\_loop`, `arm\_app\_run`)
- SEV/reset mechanism for Core1 not publicly documented
- ARM usage limited to internal ZZ9000 functions

---

#### **X1 → XX4 — early XACP experiments**

- First Amiga ↔ ARM communication tests
- Gradual discovery of DDR mapping
- Early attempts at dedicated XACP registers

- Simple memory tests
- Beginning of 68k ↔ ARM endianness issues
- First RTG corruptions caused by incorrect framebuffer offsets

---

#### **XX5 → XX6 — DDR memory validation**

- Stabilisation of usable DDR offsets
- Validation of the pipeline:  
Amiga → DDR → ARM → DDR → Amiga
- First `CMD\_MEMCPY` tests
- Validation of cache operations:  
`Xil\_DCacheInvalidateRange`, `Xil\_DCacheFlushRange`
- Discovery of the strict 32-byte alignment requirement for Cortex-A9 caches

---

#### **XX7 — first real MP3 success**

- First truly functional ARM MP3 decoding
- minimp3 integration
- First correct WAV produced by the ARM
- PCM endianness validation
- Very low 68060 CPU load (~2–4%)
- Full validation:  
MP3 → ARM decode → PCM → AHI  
This is the first truly “useful” XACP firmware.

---

#### **XX8 — XACP protocol redesign**

- Abandoned the old multi-register mapping
- Switched to a state machine:  
`0x64 = CMD/STATUS`, `0x68 = PARAM`
- Much more stable architecture
- Added: `x\_status`, `x\_result`, `x\_error`, `x\_rate`, `x\_channels`
- Definitive foundation of the modern XACP protocol

---

#### **XX9 — streaming architecture**

- Introduction of ring buffers
- Real-time MP3 streaming
- `OP\_STREAM\_OPEN`, `OP\_STREAM\_CLOSE`
- MP3 input ring, PCM output ring
- Permanent ARM decode loop on Core0
- First long streaming playback sessions  
But still: choppiness, starvation, imperfect refill.

---

#### **XX10 — streaming stabilisation**

- Fixed critical refill bugs
- Fixed uint32 underflow
- Fixed infinite decode loop
- Clean streaming WAV validation
- ARM refill validation
- MP3 leftover validation
- Streaming cache policy validation

**\*\*State:\*\*** offline decode = stable; streaming = functional but still improvable.

---

#### **XX11–XX13**

- Amiga-side MP3 ring improvements
- Reduced starvation
- Stable long-track playback

---

#### **XX14**

- Asynchronous refill refinements
- Improved PCM ring behaviour

---

#### **XX15**

- Buffering experiments
- Reduced underruns
- Lower 68k CPU load

---

#### **XX16 / XX16a / XX16b / XX16c**

- Stabilised streaming architecture
- Improved backpressure policy
- MULTI buffering strategy
- Reduced CPU usage on 68060
- Improved compatibility with `mpega.library` / AmigaAMP
- Groundwork for XACP V1 opcode reservations

---

#### **Core1 branch — 2026**

**Parallel development alongside the XX firmwares.**

Successive validations:

- Dynamic Core1 launch
- Heartbeat
- Bare-metal blobs
- DDR mailbox
- Framebuffer rendering
- Mandelbrot
- Real-time Julia
- RTG Workbench window
- Experimental double buffering

**\*\*First proof:\*\* Core1 executing real-time graphics rendering visible inside Workbench.**

---

Opcodes Present in XX10

All go through `REG\_CMD = 0x64`, with `XACP\_Command` at `fb + 0x04000000`.

---

OP\_MEMCPY = 1 — ARM memory copy with cache ops

The most basic one. The ARM copies `input\_size` bytes from `input\_offset` to `output\_offset` in DDR, with proper source cache invalidation and destination cache flush.

**\*\*What this actually does:\*\*** validates that the DDR bridge works, offsets are correct, and cache ops are clean. A permanent diagnostic tool. Used in ZZDecodeTest v1/v2 before implementing decoding.

**\*\*Result:\*\*** `result1 = bytes copied`, `status = DONE`.

---

OP\_Mandelbrot = 2 — fractal rendering by the ARM

The ARM computes a full Mandelbrot and writes the result directly to the RTG framebuffer, without the 68k touching a single pixel.

**\*\*Parameters in XACP\_Command:\*\***

...

param1 = framebuffer\_ptr (DDR destination offset in the RTG fb)  
param2 = pitch\_bytes (actual BytesPerRow of the RTG bitmap)  
param3 = width / height (packed or separate depending on version)  
param4 = max\_iter  
+ xmin/xmax/ymin/ymax in Q16.16 (fixed point, e.g. -2.5 = -163840)  
...

**\*\*What this actually does:\*\*** first pure compute offload use case. The Amiga sends parameters, the ARM renders, the Amiga displays. A validation checksum in `result1`/`result2` proves the ARM did the calculation. ARM/68060 speed ratio shown on screen.

**\*\*Status:\*\*** hardware-validated with ZZMandelbrot\_XACP, running in an Intuition RTG window on Workbench.

---

OP\_MP3\_DECODE = 3 — full offline MP3 decoding

The ARM receives a whole MP3 file in DDR, decodes it frame by frame with minimp3, and returns 16-bit signed PCM.

**\*\*Parameters:\*\***

...

input\_offset = DDR offset of the MP3 buffer (typically fb + 0x04100000)

input\_size = MP3 file size

output\_offset = DDR offset of the destination PCM buffer (fb + 0x04800000)

...

**\*\*Results:\*\***

...

result1 = PCM bytes written

result2 = sample rate (44100 / 48000)

param1 = channels (1 / 2)

...

**\*\*What this actually does:\*\*** play any MP3 file from the Amiga with 2–4% 68060 CPU. Currently used by ZZMP3Play\_XACP and mpega.library in offline mode. Limited by file size fitting in DDR in a single pass.

**\*\*Status:\*\*** hardware-validated, production.

---

### OP\_STREAM\_OPEN = 4 / OP\_STREAM\_CLOSE = 5 — real-time MP3 streaming

Full streaming pipeline with DDR ring buffers. The ARM runs a decoding loop inside the idle loop, the Amiga feeds the MP3 ring and consumes the PCM ring in parallel.

**\*\*Fixed memory layout:\*\***

...

fb + 0x04002000 → struct XACP\_StreamControl (shared state)

fb + 0x04100000 → MP3 input ring (512 KB)

fb + 0x04200000 → PCM output ring (1 MB, big-endian for AHI)

...

**\*\*What this actually does:\*\*** play MP3 files of arbitrary size, real-time AHI playback. Foundation of mpeg4.library and the future Amiga-side MP3 ring (XX11).

**\*\*Status:\*\*** hardware-validated (XX10), residual choppiness being fixed for XX11.

---

Summary — what you can do with XX10 today

| Opcode | Name            | Concrete use                        | Status                      |
|--------|-----------------|-------------------------------------|-----------------------------|
| ---    | ---             | ---                                 | ---                         |
| 1      | OP_MEMCPY       | DDR bridge / cache diagnostic       | > production                |
| 2      | OP_MANDELBROT   | ARM fractal rendering, compute demo | > hardware-validated        |
| 3      | OP_MP3_DECODE   | Offline MP3 playback, low CPU       | > hardware-validated        |
| 4      | OP_STREAM_OPEN  | Real-time MP3 streaming AHI         | > working, XX11 in progress |
| 5      | OP_STREAM_CLOSE | Clean stream close                  | > functional                |

---

**\*\*What this means for the XACP specification:\*\*** OP\_MANDELBROT proves that XACP is not just an audio protocol — it's a generic compute framework. Any bare-metal ARM workload (DSP, decompression, rendering, partial emulation) can be plugged into the same mechanism without touching the protocol architecture.

---

## Currently validated state

### **\*\*Really working:\*\***

- XACP Core0
- ARM MP3 decode
- PCM streaming
- DDR mailbox
- Core1 launch
- Core1 heartbeat
- ARM Mandelbrot
- ARM Julia
- RTG Workbench window
- Experimental double buffering

### **\*\*Not yet finalised:\*\***

- Permanent Core1 runtime
- XACP → Core1 bridge
- Direct shared framebuffer
- 68k CPU load reduction (further)

- Video streaming
- PicoDrive
- ZZUNZIP
- Stable public API

## 6. XACP V1 Opcode Map

XACP V1 introduces a structured opcode namespace intended to stabilize the protocol ABI and prepare future ARM services without requiring major firmware redesigns.

All current opcodes remain compatible with existing XX16c firmware behavior. Reserved opcodes currently return XACP\_ERR\_NOT\_IMPLEMENTED and have no side effects.

### 0x0000–0x00FF : Core / System

| Opcode | Define          | Status      | Description                  |
|--------|-----------------|-------------|------------------------------|
| 0x0001 | OP_MEMCPY       | Implemented | ARM DDR-to-DDR copy          |
| 0x0002 | OP_MANDELBROT   | Implemented | Mandelbrot / Julia rendering |
| 0x0003 | OP_MP3_DECODE   | Implemented | Offline MP3 decode           |
| 0x0004 | OP_STREAM_OPEN  | Implemented | Streaming MP3 open           |
| 0x0005 | OP_STREAM_CLOSE | Implemented | Streaming MP3 close          |

### 0x0100–0x01FF : Audio / DSP

| Opcode | Define          | Status   | Description              |
|--------|-----------------|----------|--------------------------|
| 0x0110 | OP_SID_RENDER   | Reserved | ZZSID rendering          |
| 0x0120 | OP_MIDI_SF2     | Reserved | SoundFont MIDI synth     |
| 0x0130 | OP_DSP_EQ       | Reserved | Software equalizer / DSP |
| 0x0140 | OP_PCM_RESAMPLE | Reserved | PCM resampling           |

### 0x0200–0x02FF : Image / RTG / Datatypes

| Opcode | Define              | Status   | Description                 |
|--------|---------------------|----------|-----------------------------|
| 0x0200 | OP_IMG_SCALE        | Reserved | Image scaling               |
| 0x0210 | OP_CHUNKY_TO_PLANAR | Reserved | Chunky-to-planar conversion |
| 0x0211 | OP_PLANAR_TO_CHUNKY | Reserved | Planar-to-chunky conversion |
| 0x0220 | OP_DATATYPE_DECODE  | Reserved | Datatype decode helpers     |

### 0x0300–0x03FF : Core1 / Execution

| Opcode | Define            | Status   | Description           |
|--------|-------------------|----------|-----------------------|
| 0x0300 | OP_CORE1_START    | Reserved | Launch Core1 blob     |
| 0x0301 | OP_CORE1_STOP     | Reserved | Stop Core1 execution  |
| 0x0302 | OP_CORE1_STATUS   | Reserved | Query Core1 state     |
| 0x0310 | OP_PICODRIVE_LOAD | Reserved | PicoDrive ROM launch  |
| 0x0320 | OP_DOSBOX_START   | Reserved | DOSBox runtime        |
| 0x0330 | OP_MPEG_DECODE    | Reserved | MPEG video decode     |
| 0x0340 | OP_ARM_QUAKE      | Reserved | ARM Quake experiments |

### 0x0400–0x04FF : Compression / Archive



| Opcode | Define            | Status   | Description       |
|--------|-------------------|----------|-------------------|
| 0x0400 | OP_INFLATE        | Reserved | DEFLATE inflate   |
| 0x0401 | OP_ZIP_DECOMPRESS | Reserved | ZIP decompression |
| 0x0410 | OP_LHA_EXTRACT    | Reserved | LHA extraction    |
| 0x0420 | OP_LZX_DECOMPRESS | Reserved | LZX decompression |

#### **0x0500–0x05FF : Crypto / Hash**

| Opcode | Define         | Status   | Description    |
|--------|----------------|----------|----------------|
| 0x0500 | OP_HASH_MD5    | Reserved | MD5 hash       |
| 0x0501 | OP_HASH_SHA256 | Reserved | SHA-256 hash   |
| 0x0510 | OP_AES_DECRYPT | Reserved | AES decryption |

Reserved opcodes are intentionally defined in advance to stabilize the XACP V1 ABI and prepare future extensions without changing the protocol architecture.

---

## **7. Future Directions**

Current validated XACP services already demonstrate that the ZZ9000 ARM subsystem can be used as a general-purpose Amiga coprocessor architecture rather than a simple multimedia accelerator.

Planned or experimental future directions include:

- ZZSID ARM-assisted synthesis
- MIDI SoundFont synthesis
- Software DSP / equalizer
- Datatype acceleration
- ZIP/LHA/LZX decompression
- Runtime Core1 execution services
- MPEG video decode
- PicoDrive / MegaDrive experiments
- ARM-assisted rendering and framebuffer services
- Additional DSP and multimedia workloads

The long-term objective of XACP is to progressively transform the ZZ9000 ARM subsystem into a stable coprocessor platform usable from AmigaOS while preserving a native 68k-driven Amiga environment.